



Lecture 3

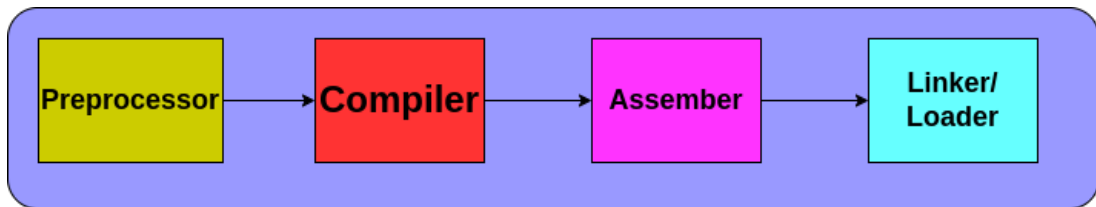
Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

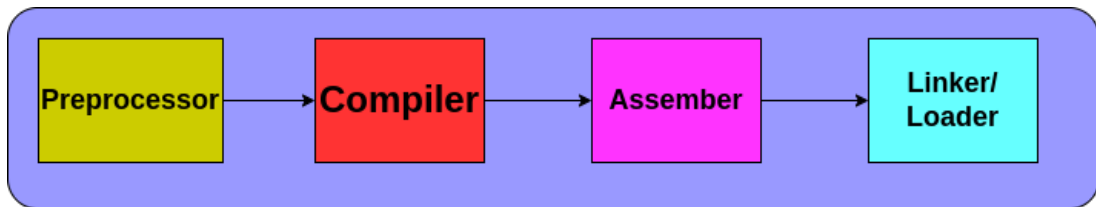
July 21, 2026

Basic flow



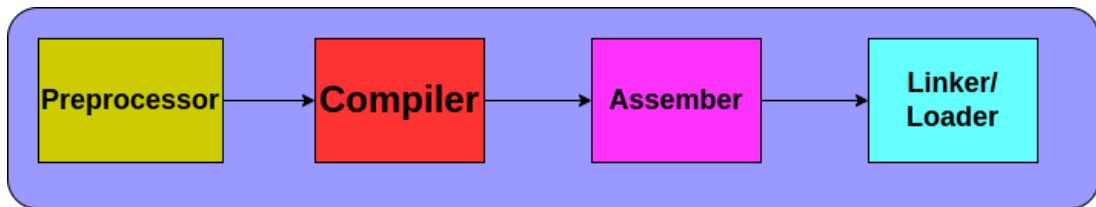
- `gcc --save-temps fileName.c`

Basic flow



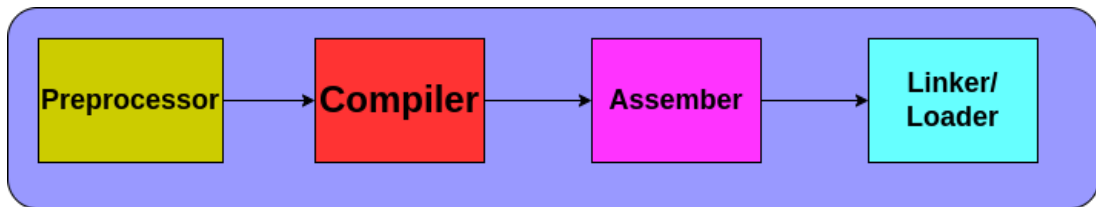
- `gcc --save-temps fileName.c`
- Preprocessing: includes header file and macro expansion.

Basic flow



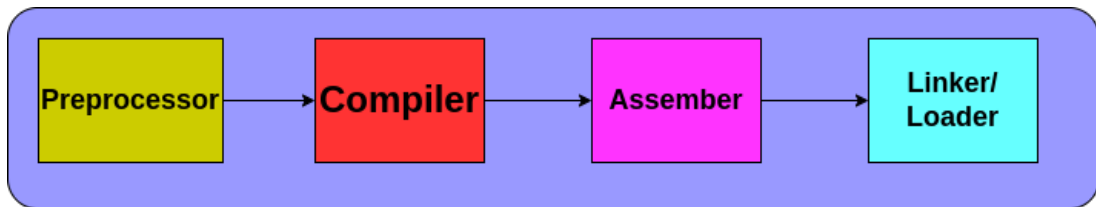
- `gcc --save-temps fileName.c`
- Preprocessing: includes header file and macro expansion.
- Compiler: generate assembly file

Basic flow



- `gcc --save-temps fileName.c`
- Preprocessing: includes header file and macro expansion.
- Compiler: generate assembly file
- Assembler: generate the relocatable object file

Basic flow



- `gcc --save-temps fileName.c`
- Preprocessing: includes header file and macro expansion.
- Compiler: generate assembly file
- Assembler: generate the relocatable object file
- Linker/loader: link different object files into a single binary. Load it to the main memory.

How to translate easily

- Translate in steps. Each step handles a reasonably simple, logical, and well-defined task

How to translate easily

- Translate in steps. Each step handles a reasonably simple, logical, and well-defined task
- Design a series of program representations

How to translate easily

- Translate in steps. Each step handles a reasonably simple, logical, and well-defined task
- Design a series of program representations
- Intermediate representations should be amenable to program manipulation of various kinds (type checking, optimization, code generation etc.)

How to translate easily

- Translate in steps. Each step handles a reasonably simple, logical, and well-defined task
- Design a series of program representations
- Intermediate representations should be amenable to program manipulation of various kinds (type checking, optimization, code generation etc.)
- Representations become more machine-specific and less language-specific as the translation proceeds

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language
- The first step is recognizing/knowing alphabets of a language. For example

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language
- The first step is recognizing/knowing alphabets of a language. For example
 - ▶ English text consists of lower and upper case alphabets, digits, punctuations and white spaces

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language
- The first step is recognizing/knowing alphabets of a language. For example
 - ▶ English text consists of lower and upper case alphabets, digits, punctuations and white spaces
 - ▶ Written programs consist of characters from the ASCII characters set (normally 9-13, 32-126)

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language
- The first step is recognizing/knowing alphabets of a language. For example
 - ▶ English text consists of lower and upper case alphabets, digits, punctuations and white spaces
 - ▶ Written programs consist of characters from the ASCII characters set (normally 9-13, 32-126)
- The next step to understand the sentence is recognizing words (lexical analysis)

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language
- The first step is recognizing/knowing alphabets of a language. For example
 - ▶ English text consists of lower and upper case alphabets, digits, punctuations and white spaces
 - ▶ Written programs consist of characters from the ASCII characters set (normally 9-13, 32-126)
- The next step to understand the sentence is recognizing words (lexical analysis)
 - ▶ English language words can be found in dictionaries

The first few steps

- The first few steps can be understood by analogies to how humans comprehend a natural language
- The first step is recognizing/knowing alphabets of a language. For example
 - ▶ English text consists of lower and upper case alphabets, digits, punctuations and white spaces
 - ▶ Written programs consist of characters from the ASCII characters set (normally 9-13, 32-126)
- The next step to understand the sentence is recognizing words (lexical analysis)
 - ▶ English language words can be found in dictionaries
 - ▶ Programming languages have a dictionary (keywords etc.) and rules for constructing words (identifiers, numbers etc.)

Lexical Analysis

- Recognizing words is not completely trivial. For example:

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are
- The language must define rules for breaking a sentence into a sequence of words.

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are
- The language must define rules for breaking a sentence into a sequence of words.
- Normally, white spaces and punctuations are word separators in languages.

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are
- The language must define rules for breaking a sentence into a sequence of words.
- Normally, white spaces and punctuations are word separators in languages.
- In programming languages, a character from a different class may also be treated as word separator.

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are
- The language must define rules for breaking a sentence into a sequence of words.
- Normally, white spaces and punctuations are word separators in languages.
- In programming languages, a character from a different class may also be treated as word separator.
- The lexical analyzer breaks a sentence into a sequence of words or tokens:

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are
- The language must define rules for breaking a sentence into a sequence of words.
- Normally, white spaces and punctuations are word separators in languages.
- In programming languages, a character from a different class may also be treated as word separator.
- The lexical analyzer breaks a sentence into a sequence of words or tokens:
 - ▶ `if a == b then a = 1 ; else a = 2 ;`

Lexical Analysis

- Recognizing words is not completely trivial. For example:
ist his ase nte nce?
- Therefore, we must know what the word separators are
- The language must define rules for breaking a sentence into a sequence of words.
- Normally, white spaces and punctuations are word separators in languages.
- In programming languages, a character from a different class may also be treated as word separator.
- The lexical analyzer breaks a sentence into a sequence of words or tokens:
 - ▶ `if a == b then a = 1 ; else a = 2 ;`
 - ▶ Sequence of words (total 14 words)
"if", "a", "==", "b", "then", "a", "=", "1", ";", "else", "a", "=", "2",
";",